Redrob Candidate Ranker

Ranking candidates the way a great recruiter would

Top-100 of a 100,000 candidate pool — demonstrated on the Senior AI Engineer JD.
JD-adaptive: works for any role. Reads careers, not keywords. CPU-only, ~2 min for 100K.

Team: YOUR_TEAM_NAME

Hybrid interpretable ranker · TF-IDF + structured fit + behavioral gates

THE PROBLEM

Keyword filters can't see what matters

- Recruiters skim hundreds of profiles and still miss the right person — keyword filters reward the wrong signals.
- The dataset is built to PUNISH keyword matchers:
- Keyword stuffers — an Accountant whose skills list is full of 'NLP, LLM, FAISS'.
- Plain-language Tier-5s — built a real recsys at a product company, never says 'RAG'.
- ~80 honeypots — subtly impossible profiles (8 yrs at a 3-yr-old company).
- The provided sample_submission.csv falls for it: HR Managers & Content Writers at rank 1–2.

A naive ranker outputs:

#1 HR Manager

#2 Content Writer

#3 Graphic Designer

#4 Accountant

…all with 9 'AI skills'

= exactly the trap.

KEY INSIGHT

The signal is the career, not the skills list

- Skills[] is randomly stuffed noise — easy to fake. We mostly ignore it.
- We read the PROSE: current title, every past title, and the career-history descriptions.
- A 'Marketing Manager' with a perfect AI skill list is capped low — the role is the anti-stuffer signal.
- A generic 'Software Engineer' whose descriptions show ranking/retrieval work is LIFTED toward ML.
- Behavioral signals decide who is actually hireable, not just good on paper.

✗ Keyword stuffer

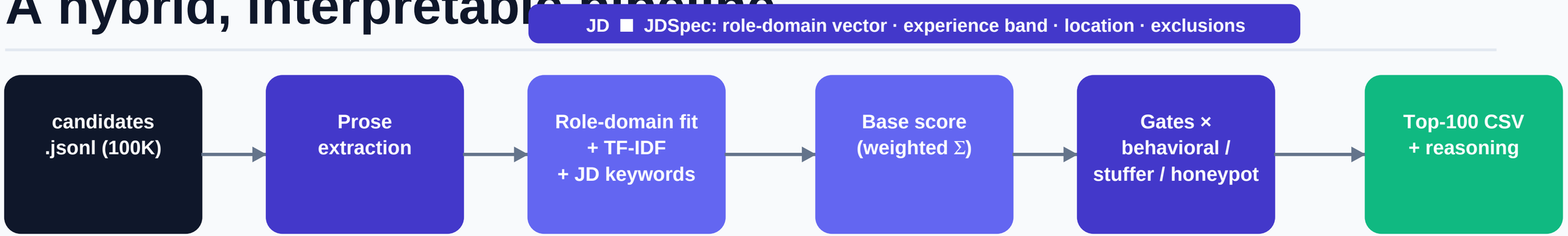
Title: Accountant
Skills: NLP, LLM, FAISS, RAG…
Prose: ledgers, GST, audits
→ **capped + stuffer penalty**

✓ Real fit

Title: Recommendation Sys Eng
Prose: built ranking at scale,
FAISS index, NDCG eval, prod
→ **ranks #1**

ARCHITECTURE

A hybrid, interpretable pipeline



- JD-adaptive: paste any JD — role domain, experience band, location and excluded specialties are parsed at run time. AI JD -> ML engineers; marketing JD -> marketers.
- Role fit = cosine of the candidate's domain vector (from titles + prose, never the stuffable skills list) vs the JD's.
- Same pipeline powers the submission CSV and the Streamlit sandbox. TF-IDF by default (zero network); optional embedding booster.

SCORING MODEL

Weighted base, then multiplicative gates

```
base  = 0.36·role_fit + 0.18·semantic + 0.16·evidence
        + 0.10·experience + 0.10·company_type + 0.10·location

final = base × behavioral × verified × stuffer × excl-specialty × hopping × honeypot
```

Role / domain fit	`0.36`	candidate-vs-JD domain cosine
Semantic (TF-IDF)	`0.18`	prose vs JD, not skills[]
JD-keyword evidence	`0.16`	JD's salient terms in the prose
Experience	`0.10`	parsed band, peak at centre
Company type	`0.10`	product > services (tech roles)
Location	`0.10`	parsed JD city/country

TRAP DEFENSE

Four multiplicative gates

Keyword-stuffing

Many JD-keyword skills + weak role fit → penalty up to –45%.

Honeypot / consistency

Role longer than whole career, 'expert' in skills with 0 months used, impossible date spans → forced out of top 100.

Excluded specialty (from JD)

The JD's 'do NOT want' section is parsed; a title that matches an excluded specialty (e.g. computer vision for this role) → down-weighted.

Behavioral availability

Low response rate, inactive 6+ months, long notice → down-weighted. A perfect-on-paper unreachable candidate is not hireable.

RESULTS

Top-100 on the released pool

100 / 100
India candidates

0 (limit 10)
Honeypots in top-100

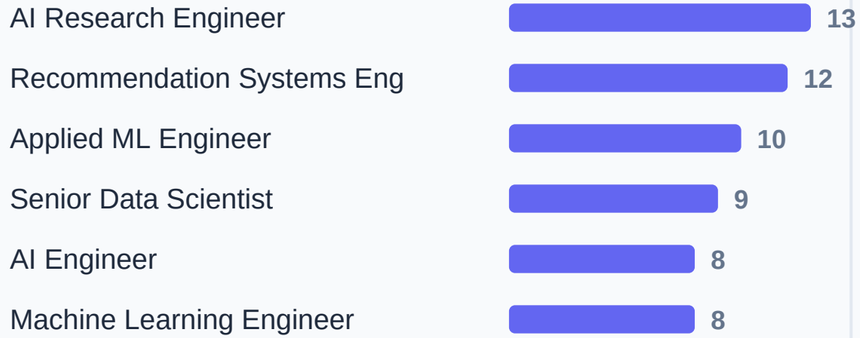
54
In 6–8y ideal band

1.00 → 0.87
Score range

Top 5 picks

RANK	TITLE	YRS	SCORE
#1	Senior Machine Learning Engineer	7.2	1.000
#2	Senior Machine Learning Engineer	6.1	0.950
#3	Recommendation Systems Engineer	8.0	0.948
#4	AI Research Engineer	4.9	0.943
#5	Senior NLP Engineer	7.8	0.943

Top-100 role mix (all genuine ML/IR)



REASONING

Grounded, specific, varied

- Every reasoning string is assembled ONLY from facts in the candidate's own profile — no hallucinated skills.
- Tone adapts to rank; honest concerns (low response rate, long notice, services-heavy background) are surfaced.
- 99 / 100 reasonings are unique — no templated 'insert name' filler.

#1	Senior ML Engineer, 7.2 yrs; retrieval/ranking work, eval-metric experience, production signals; Pune. Responsive (95%).
#7	Applied ML Engineer, 6.0 yrs; retrieval/ranking + production signals; Kolkata. Concern: long notice period (90d).
#84	Junior ML Engineer, 6.8 yrs; production/scale signals, product-company background; Jaipur.

COMPUTE & REPRODUCIBILITY

Built for a real production budget

Runtime ≤ 5 min	~2 min for 100K (TF-IDF path)
Memory ≤ 16 GB	well under — sparse TF-IDF + streamed JSONL
CPU only, no GPU	yes, everywhere
Network off	zero API / LLM calls during ranking
One command	python rank.py --candidates … --out …
Passes validator	validate_submission.py → 'Submission is valid.'
Sandbox	Streamlit app runs the same pipeline on a sample

What we'd ship next

- Dense embeddings (BGE/E5) as the primary retriever — booster already wired in, TF-IDF as fallback.
- Learning-to-rank (XGBoost) trained on recruiter-engagement labels once available.
- Proper offline eval harness: NDCG / MRR / MAP with bootstrapped confidence intervals.
- Online A/B testing + recruiter-feedback loop to close the offline→online gap.

Reads careers, not keywords · CPU-only · fully reproducible

GitHub: github.com/YOUR_USERNAME/redrob-ranker · Sandbox: Streamlit Community Cloud